Gradient-Based Physics-Informed Neural Network

Kirti Beniwal and Vivek Kumar

Abstract Physics-informed neural networks (PINNs) are one of the most effective tool of deep learning used for solving differential equation. However, one of the significant drawbacks of PINN is its low accuracy in the initial stage, even with a large number of training points. This study introduces gradient-enhanced physics-informed neural networks (gPINNs) that are trained for solving partial differential equations. Here, the proposed approach is contrasted with a physics-informed neural network, which has proven to be an efficient technique for resolving forward and inverse PDE issues. PINN increases the performance of neural networks (NNs) by using the physical information contained in PDE as a regularization term. So, gradient-based physics-informed neural network (gPINNs) is introduced here to improve the performance of PINNs. gPINNs control the gradient information of equation error and embed this term into the loss term. This method is tested for various PDEs and found to be more effective than PINN. Further, gPINN was implemented with the residual-based adaptive refinement (RAR) method to enhance or increase wave equation accuracy. Finally, we used the Keras library in PINN and gPINN algorithm to find the unknown parameter of inverse PDE.

Keywords Partial differential equation · Physics-informed neural network · Residual-based adaptive refinement

1 Introduction

While deep learning (DL) has demonstrated remarkable successes in a variety of applications, it has not been widely used to solve partial differential equations (PDEs) until recently discussed by Ex. Karniadakis et al. [1, 2]. While artificial neural network (ANN) was introduced in an article by Ex. McCulloch and Pitts [2, 3], as it was not so popular at that time because of computational machines. This network is made

K. Beniwal · V. Kumar (✉)
Department of Applied Mathematics, Delhi Technological University,
Bawana Road, Delhi, India
e-mail: vivekkumar.ag@gmail.com

S. Kumar et al. (eds.), *Third Congress on Intelligent Systems*, Lecture Notes in Networks and Systems 613, https://doi.org/10.1007/978-981-19-9379-4_54

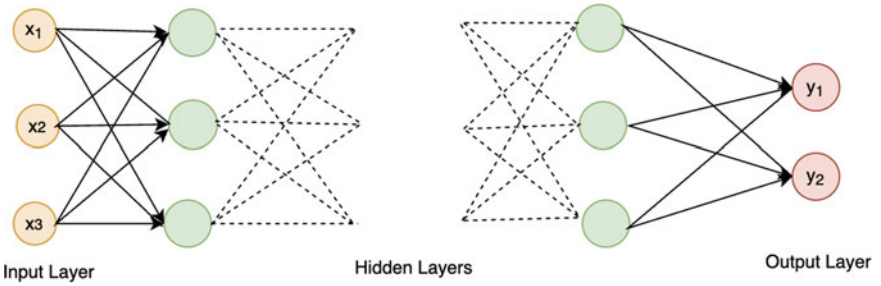

Fig. 1 Graph showing a feedforward neural network (FNN), in which one or more artificial neurons are placed into an input layer, one or more hidden layers, and one or more output layers

up of input, output, and hidden layers with several number of neurons. The ANN network with more than one hidden layer is called deep neural network (DNN). A lot of attention has been paid to neural networks (NNs) in solving differential equations in recent years. Their universal approximation properties provide an alternative approach to solving differential equations by providing a nonlinear approximant via a variety of network structures and activation function. Classification, pattern recognition, and regression have all been revolutionized by deep learning. There are various numerical techniques to solve differential equations discussed by Ex. Lochab and kumar [4–8], such as finite element method [9–11] and finite difference method [12, 13]. A new class of neural networks was introduced called **Physics-Informed Neural Network (PINN)**. It is specially designed to solve ordinary differential equations (ODEs), partial differential equations (PDEs), integro-differential equations, etc. It is used not only to solve differential equations but also to predict the solution of the equation from the data called 'Data Driven Discovery' given by Ex. Raissi et al. [14]. The main advantage of PINN is that it is mesh-free [15, 16] and simple; therefore, it has been successfully implemented to solve various problems in different field of science such as fluid mechanics and biomedicine.

In the past decades, various types of NN have been introduced such as convolutional neural network (CNN), recurrent neural network (RNN), but the simplest one is feedforward neural network (FNN). In this paper, FNN and residual neural network have been implemented, for solving PDE.

Consider a NN with L number of layers and $L-1$ hidden layers. N_l denotes the neurons in lth layer, (N_o = input layer neurons) and (N_L = output layer neurons). Let W^l, b^l denotes weight matrix and bias vector, respectively, at the lth layer (Fig. 1). For any nonlinear activation function σ , the feedforward neural network is defined as follow [17]:

input layer : $\mathbf{N}^0(x) = x$

hidden layer : $\mathbf{N}^l(x) = \sigma(W^l\mathbf{N}^{l-1}(x) + b^l)$, for $1 \leq l \leq L-1$,

output layer: $\mathbf{N}^L(x) = W^L\mathbf{N}^{L-1}(x) + b^L$

In most of the cases, tanh, sigmoid, and rectified linear units (ReLUs) are used as activation functions. Furthermore, it is required to compute the derivatives of output with respect to input. In deep learning framework, automatic differentiation is used to

compute such derivatives using backpropagation. The basic formulation of solving PDE using PINN requires an loss function which has to be minimized via automatic differentiation. Usually, minimization of the loss function was trained using gradient-based optimizers like Adam, Limited- Broyden-Fletcher-Goldfarb-Shanno algorithm (L-BFGS), and gradient decent. The differentiation using automatic differentiation can be conveniently done with machine learning libraries like Ex. Abadi et al. [18] presented a detailed study of Tensorflow and Ex Paszke et al. [19] discussed PyTorch.

It is still necessary to address some issues with PINNs, despite promising results. The problem of improving the accuracy and efficiency of PINNs remains an open one. There are still some aspects of PINN that can be improved such as during training the residual points are randomly distributed while there are some other methods also of training and sampling that use same number of training points, which gives better accuracy, i.e., RAR. Also, in PINN, it becomes difficult to balance the loss terms as it contains various loss terms corresponds to PDE and PDE residuals. So, for that domain decomposition method can be used for large domain.

In PINN, neural networks were trained to minimize the loss function obtained using PDE residuals, and therefore, the PDE residual is used as losses. Since this concept is simple and employed by many researchers in the field, no attention is currently being paid to other types of losses connected to PDEs. As a result, it can be clearly observed that if PDE residual becomes zero, it must also need to be zero in terms of the gradient discussed by Lu et al. [20].

By holding gradient part of the PDE residuals, a method is developed, i.e., gradient-enhanced PINN (gPINN), which improves the accuracy of PINNs by using gradient-enhanced loss functions. Using neural networks and the Gaussian process [21], it has been shown that the general concept of using gradient information is advantageous for function regression. These approaches, however, were mainly constructed for function regression and do not work for solving forward or inverse PDEs. Instead, they use function gradients in addition to function values. The sole method in this area that employs the input's loss gradient as an additional loss is input gradient regularization. Hence, a fundamentally different approach has been provided to these prior approaches, as the PDE residual's gradient applied to the models inputs. Further improving performance is achieved by combining gPINN with the RAR method.

This paper is organized as follows: In Sect. 2, formulation of PINN, gPINN, and gPINN with RAR is discussed. In Sect. 3, forward equation solved using gPINN+RAR and a inverse problem using keras library. Then, summary of this study is given in Sect. 4.

2 Methods

This section includes a brief overview of PINN and gradient-enhanced physics-informed neural network (gPINN). The relevant algorithms are introduced.

2.1 PINN Algorithm for Solving Forward and Inverse PDEs

Consider a PDE that is defined on the domain Ω and boundary conditions are defined on $\partial\Omega$.

$$F\left(x; \frac{\partial y}{\partial x_1}, ...; \frac{\partial^2 y}{\partial x_1 \partial x_1}, ..., \frac{\partial^2 y}{\partial x_1 \partial x_n}\right) = 0, \quad x = (x_1, x_2, ..., x_n) \in \Omega \tag{1}$$

$$B[y](x) = 0, \quad x \in \partial\Omega \tag{2}$$

here, y is defined as unknown solution. B denotes boundary condition, initial conditions are treated in the same process as above boundary conditions. F can be linear or nonlinear differential operator ($\frac{\partial}{\partial x}$, $\frac{\partial^2}{\partial x^2}$, $y.\frac{\partial}{\partial x}$, etc). In PINN algorithm , firstly, construct a neural model to approximate the solution $y(x)$ and this NN solution is then denoted as $\widehat{y}(x; \theta)$, where x is taken as input with parameters θ and outputs a vector of same dimension as $y(x)$. Generally, there are three neural network (NN) layers: input, hidden, and output layer. In this model, assume $L - 1$ hidden layer and one input and a single output layer. During the creation of the neural network, each hidden layer receives its output from the previous one layer, and each hidden layer is composed of N_k neurons. Each layer's weight matrix and bias vector are contained in the parameter θ. During the training phase, these parameters will continully be optimized (Fig. 2). Therefore, specify the training constraints T_d and T_b for the PDE and boundary/initial conditions to train the model. Let T_d denotes the data points in the domain and T_b be the set of boundary points. In a PINN, the loss function is defined as:

$$J(\theta, T) = w_d J_d(\theta; T_d) + w_b J_b(\theta; T_b) \tag{3}$$

where,

$$J_d(\theta; T_d) = \frac{1}{|T_d|} \sum_{x \in T_d} |\widehat{y}^i - y(x_y^i, t_y^i)|^2 \tag{4}$$

$$J_b(\theta; T_b) = \frac{1}{|T_b|} \sum_{x \in T_b} |B(\widehat{y}, x)|^2 \tag{5}$$

where w_d and w_b are the weights, and $T = (x_1, x_2, ..., x_{|T|})$ is training set of size $|T|$. T_d and T_b denotes the set of residual points of the training set. Here, $y(x_y^i, t_y^i))$ is training data from initial and boundary conditions. The above loss function contains partial and normal derivative which is then handled via automatic differentiation (AD). In last step, train the NN by minimizing the loss function $J(\theta; T)$ to find the best parameters θ. As this function is nonlinear, so this will be minimized using gradient-based optimizers like, Adam [22] and L-BFGS [22].

$$\theta_{n+1} = \theta_n - \beta \bigtriangledown_\theta J(\theta_n)$$

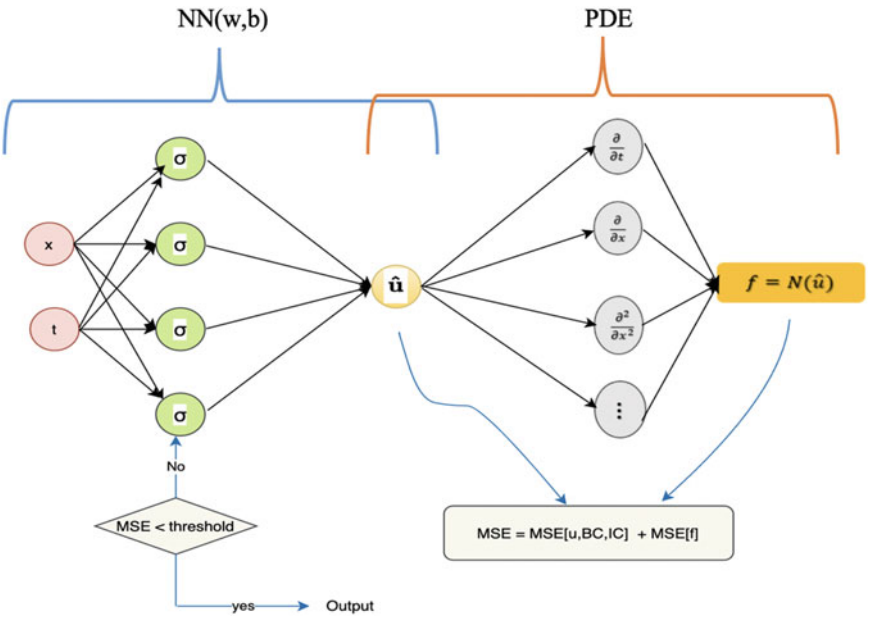

Fig. 2 An illustration of the physics-informed neural network, used to solve partial differential equations [17]

PINNs have the advantage of being able to be used for forward problems and for inverse PDE-based problems also. If there is any unknown parameter α in Eq. (1), then add an additional loss term of y on the set of points T_i in Eq. (3). So, a new loss function for inverse problem is defined as

$$J(\theta, T) = w_d J_d(\theta; T_d) + w_b J_b(\theta; T_b) + w_i J_i(\theta, \alpha; T_i). \tag{6}$$

where,

$$J_i(\theta, \alpha; T_i) = \frac{1}{|T_i|} \sum_{x \in T_i} \left(|\widehat{y} - y(x)|^2 \right) \tag{7}$$

2.2 *Gradient-Based Physics-Informed Neural Networks (gPINNs)*

In PINN, the PDE residuals are only enforced to be zero; since $f(x) = 0$ for any x, its derivatives should also be zero [20]. In this case, suppose that the PDE solution is smooth enough for $f(x)$ to have a gradient of PDE residual, and then, gPINN is introduced to make the derivative of PDE residual to be zero .

$$\nabla F(x) = \left(\frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2}, ..., \frac{\partial y}{\partial x_n} \right) = 0, \quad x \in \Omega$$

Therefore, the loss function is defined as:

$$J = w_d J_d + w_b J_b + w_i J_i + \sum_{i=1}^{n} w_{fi} J_{fi}(\theta; T_{fi}), \tag{8}$$

where

$$J_{fi}(\theta; T_{fi}) = \frac{1}{|T_{fi}|} \sum_{x \in T_{fi}} \left| \frac{\partial y}{\partial x_i} \right|^2 \tag{9}$$

Here, T_{fi} denotes set of residual points for the derivative term. In this study, grid search is used to determine the optimal values of weights; we choose $w_{f_1} = w_{f_2} = ... = w_{f_n}$. When gPINN is tested on some PDEs, the performance is sensitive to the weight value, while on others it is not. As this will be demonstrated in numerical examples, that how, gPINN improves the estimation of PINN by imposing the gradient of PDE residual. As a result, it is possible to predict the solutions for u with more accuracy and with fewer training points. The gPINN algorithm also enhances the accuracy of predictions for $\frac{\delta u}{\delta x_i}$. Motivation of gPINN is that usually the PDE residual of PINN fluctuates around zero, and penalizing the slope of the residual would lessen this fluctuation and bring the residual closer to zero.

2.3 Residual-Based Adaptive Refinement (RAR)

As residual points are randomly distributed in the domian of PINN, a residual-based adaptive refinement (RAR) [20] method is created to optimize the distribution of residual points throughout the training phase. To increase the model's precision and effectiveness during training, add extra residual points when the PDE residual is high. Steps for gPINN with RAR algorithm:

1. For certain number of iterations, train the NN using gPINN.
2. Find the PDE residual at random points in the domain.
3. To the training set T, add n new points that has largest residuals.
4. In this step, repeat Steps 1, 2, and 3 until a threshold being reached, or until mean residual falls below that threshold.

3 Results

In section, forward and inverse PDE will be discussed using gradient-enhanced PINN (gPINN) and gPINN with RAR to improve the accuracy. Equations are constructed

using neural network models that will be based on the provided initial and boundary condition. To test gradient-enhanced PINN, approximation results will be compared with true solutions and PINN results.

3.1 Function Approximation Using NN and gNN

First, the benefits of adding gradient information are demonstrated through a pedagogical example. So, consider a function.

$$y(x) = \sin(10x^2), \quad x \in [0, 1] \tag{10}$$

Based on the training dataset $(x_1, y(x_1))$, $(x_1, y(x_1))$, ..., $(x_n, y(x_n))$, where $(x_1, x_2, ..., x_n)$ are equispaced points in a sample [0, 1]. In order to train a neural network, a loss function is used as follows:

$$J = \frac{1}{n}\sum_{i=1}^{n} |y(x_i) - \widehat{y}(x_i)|^2 \tag{11}$$

Additionally, consider the gNN with the additional gradient loss function as [22].

$$J = \frac{1}{n}\sum_{i=1}^{n} |y(x_i) - \widehat{y}(x_i)|^2 + w_g \frac{1}{n}\sum_{i=1}^{n} |\bigtriangledown y(x_i) - \bigtriangledown\widehat{y}(x_i)|^2 \tag{12}$$

Using DeepXDE [20] with TensorFlow, 1 backend library code for this function was implemented. In this example, hyperbolic tangent is used as the activation operator, and hyperparameters that are used for this example are as follows: 'Adam' optimizer as it is much more efficient and provides much higher performance because it combines two gradient descent approaches: Momentum and root mean square propagation (RMSP); 'Glorot uniform' initilizer; 10 hidden layers and 1000 iterations were performed. Using different weight values w_g including 1, 0.1, 0.01, 0.001, training of the network was done for all weights values, and it was found that w_g does not affect gNN accuracy. NN and gNN shows smaller L^2 relative error even when number of training points increased; here, 20 training points were taken. The gNN algorithm has smaller error as compared to neural network which is about 1 order of magnitude more accuracy. Figure 3 shows the prediction of y and $\frac{dy}{dx}$ using 20 training points.

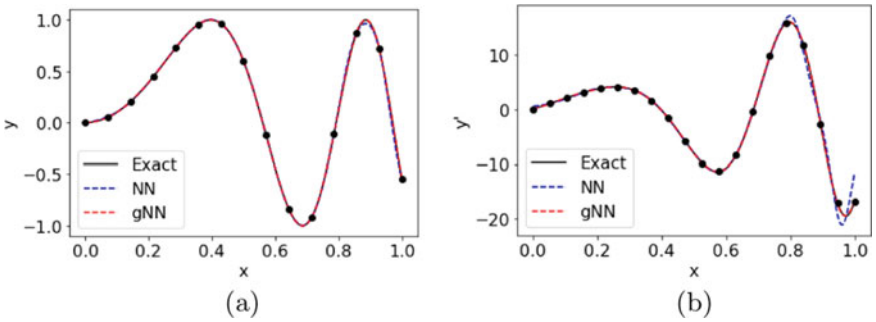

Fig. 3 Comparison between exact, NN and gNN, **a** and **b** example of predicted y and $\frac{dy}{dx}$, respectively. The black dots represent location of training points

3.2 Forward Problem Using gPINN

As a result of showing how effectively gradient loss can be added to function approximation, gPINN is applied to the solution of PDEs.

3.2.1 Wave Equation

There are numerous applications of the wave equation [23, 24] in physics, such as the study of how waves propagate through space and radio communications. It appears in many fields of science, such as the study of acoustic wave propagation, radio communications, and seismic wave propagation [25–27]. So, wave equation is mathematically expressed as:

$$y_{tt}(x,t) - c^2 y_{xx} = 0, \quad x \in (0,1), t \in (0,1) \tag{13}$$

where y is function of x (spatial variables) and t (time). The initial and boundary conditions are given as follows:

$$y(0,t) = y(1,t) = 0, \quad t \in [0,1] \tag{14}$$

$$y(x,0) = \sin(\pi x) + \sin(a\pi x), \quad x \in [0,1] \tag{15}$$

$$y_t(x,0) = 0, \quad x \in [0,1] \tag{16}$$

The solution of above equation is

$$y(x,t) = \sin(\pi x)\cos(c\pi t) + \sin(a\pi x)\cos(ac\pi t)$$

with respect to the spatiotemporal domain, we specifically treat the initial condition (15) as a special boundary condition. Equations (14) and (15) can be summerized as:

$$f(x) = \sin(\pi x) + \sin(a\pi x), \quad x \in \partial\Omega$$

To train the network, minimize the loss function defined as:

$$J(\theta) = J_y(\theta) + J_{y_t}(\theta) + J_r(\theta) \tag{17}$$

$$= \frac{1}{N_y}\sum_{i=1}^{N_y} |y(x_y^i, \theta) - f(x_y^i)|^2 + \frac{1}{N_{y_t}}\sum_{i=1}^{N_{y_t}} |y_t(x_{y_t}^i, \theta)|^2 \tag{18}$$

$$+ w_g \frac{1}{N_r}\sum_{i=1}^{N_r} \left| \frac{\partial^2 y_\theta}{\partial t^2}(x_r^i) - c^2 \frac{\partial^2 y_\theta}{\partial x^2}(x_r^i) \right|^2 .$$

where $a = 2$ and $c = 10$ and set the batch sizes to $N_y = N_{y_t} = N_r = 360$ and each time gradient descent iterates, all data points $\{x_y^i, f(x_y^i)\}_{i=1}^{N_y}$, $\{x_{y_t}^i\}_{i=1}^{N_{y_t}}$ and $\{x_r^i\}_{i=1}^{N_r}$ are uniformly selected from the relevant regions in the computational domain. 1000 residual points were considered and used hard parameters constraints for both the algorithms, i.e., PINN and gPINN. The prediction and error of these algorithms are shown in Fig. 4. The results and solutions of gPINN are more accurate as compared to that of PINN in the whole domain.

3.3 Inverse Problem Using gPINN

Aside from solving forward PDE problems, gPINN is also applied for solving inverse PDE problems. The loss term for inverse problem is discussed in Sect. 2. Similarly, gPINN algorithm is also used to solve inverse PDE.

3.3.1 Eikonal Equation

Eikonal equation is first-order nonlinear PDE that is basically confronted in wave propagation. This example shows how to solve partial differential equations [28, 29] with unknown parameters λ using the PINN and gPINN methods. Therefore, consider time-dependent Eikonal's parabolic equation over domain $D = [-1, 1]$

$$\begin{aligned} -\partial_t y(t,x) + |\nabla y(t,x)| &= \lambda^{-1}, && (t,x) \in [0,T) \times [-1,1], \\ y(T,x) &= 0, && x \in [-1,1] \\ y(t,-1) = y(t,1) &= 0, && t \in [0,T). \end{aligned}$$

λ is the unknown parameter. The solution is

$$y^*(t, x) = \lambda^{-1} \min\{1 - t, 1 - |x|\}.$$

whenever the Eikonal equation runs backward in time, it is consistent with the interpretation that it is the optimality condition for controlling systems. The change of variables $t^* = T - t$ can transform this equation into a forward evolution problem. Our final model was based on a Leaky ReLU activation function with $r = 0.1$ slope parameter, which consisted of only two hidden layers, ReLU is defined as:

$$\sigma(z) = \begin{cases} x & \text{if} \quad r \geq 0 \\ rx & \text{otherwise} \end{cases}$$

For Eiklon equation, 500 uniformly distributed training points were taken into consideration. Next, a new network class was added which include additional λ. The model was then trained for 1000 epoch which takes 30 s. Figure 5 shows the solution of Eiklon equation $y_\theta(t, x)$. This shows that the relative error calculated using

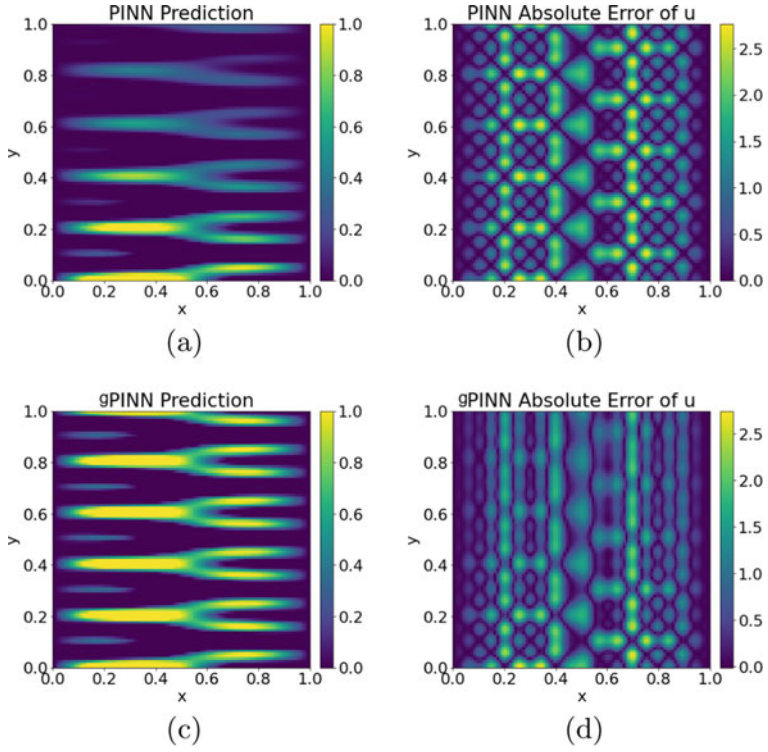

Fig. 4 Comparison between PINN and gPINNs predicted value and the absolute value of y for 1D wave equation

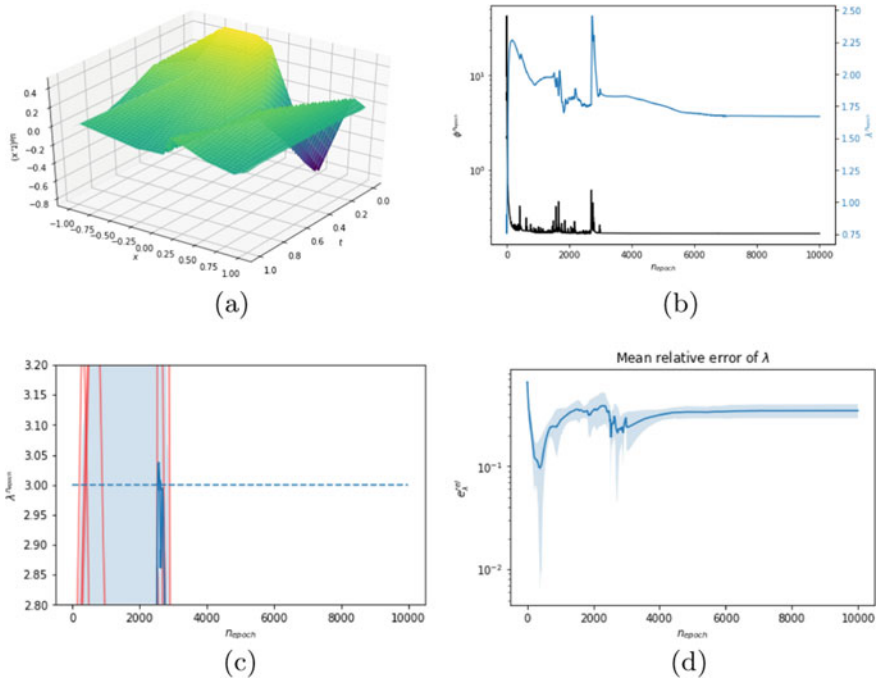

Fig. 5 **a** The solution of Eiklon equation; **b** evolution of loss values and estimated parameter; **c** evolution of λ, its mean (blue) and one standard deviation (shaded blue), true value of λ (dashed blue); **d** mean relative error

gPINN model (4.4365×10^{-6}) is less than PINN model (6.3427×10^{-4}) of identified paramter λ. Further, the evolution of λ ,true value of λ, together with its mean and standard deviation is shown below.

4 Conclusion

In this study, a method for solving partial differential equation, i.e., physics-informed neural network was introduced (PINN) and its new version gradient-enhanced PINN which improves the exactness of PINN. All the examples show that gPINN clearly gives better accuracy and outperforms PINN for relative errors of the solution and derivatives with the same number of data points. Also, for inverse problem, gPINN takes in the unknown parameter more precisely in less computational time.

References

1. Karniadakis GE, Kevrekidis IG, Lu L, Perdikaris P, Wang S, Yang L (2021) Physics-informed machine learning. Nat Rev Phys 3(6):422–440
2. McCulloch WS, Pitts W (1943) A logical calculus of the ideas immanent in nervous activity. Bull Math Biophys 5(4):115–133
3. Saraswat M, Sharma H, Balachandran K, Kim JH, Bansal JC (2021) Congress on intelligent systems. In: Proceedings of CIS 2021, vol 1
4. Kumar V, Rao SVR (2008) Composite scheme using localized relaxation with non-standard finite difference method for hyperbolic conservation laws. J sound vib. 311(3-5):786–801
5. Lochab R, Kumar V (2021) A new reconstruction of numerical fluxes for conservation laws using fuzzy operators. Int J Num Methods Fluids 93(6):1690–1711
6. Lochab Ruchika, Kumar Vivek (2021) An improved flux limiter using fuzzy modifiers for Hyperbolic Conservation Laws. Math Comput Simul 181:16–37
7. Lochab R, Kumar V (2022) A comparative study of high-resolution methods for nonlinear hyperbolic problems. ZAMM-J Appl Math Mech/Zeitschrift für Angewandte Mathematik und Mechanik: e202100462
8. Saraswat M, Sharma H, Balachandran K, Kim JH, Bansal JC () Congress on intelligent systems. In: Proceedings of CIS 2021, vol 2
9. Khari K, Kumar V (2022) An efficient numerical technique for solving nonlinear singularly perturbed reaction diffusion problem. J Math Chem 60:1356–1382. https://doi.org/10.1007/s10910-022-01365-4
10. Khari K, Kumar V (2022) Finite element analysis of the singularly perturbed parabolic reaction-diffusion problems with retarded argument. Numer Methods Partial Differ Eq 38:997–1014. https://doi.org/10.1002/num.22785
11. Sharma H, Saraswat M, Kumar S, Bansal JC (2020) Intelligent learning for computer vision. In: Proceedings of congress on intelligent systems
12. Kumar V, Mehra M (2007) Wavelet optimized finite difference method using interpolating wavelets for solving singularly perturbed problems. J Wave Theory Appl 1(1):83–96
13. Wang S, Wang H, Perdikaris P (2021) On the eigenvector bias of fourier feature networks: from regression to solving multi-scale pdes with physics-informed neural networks. Comput Methods Appl Mech Eng 384:113938
14. Raissi M, Perdikaris P, Karniadakis GE (2017) Physics informed deep learning (part i): data-driven solutions of nonlinear partial differential equations. arXiv preprint arXiv:1711.10561
15. Kumar V, Srinivasan B (2019) A novel adaptive mesh strategy for singularly perturbed parabolic convection diffusion problems. Differ Equ Dyn Syst 27(1):203–220
16. Sharma H, Saraswat M, Yadav A, Kim JH, Bansal JC (2020) Congress on intelligent systems. In: Proceedings of CIS 2020, vol 1
17. Guo Y, Cao X, Liu B, Gao M (2020) Solving partial differential equations using deep learning and physical constraints. Appl Sci 10(17):5917
18. Abadi M, Barham P, Chen J, Chen Z, Davis A, Dean J, Devin M et al. (2016) TensorFlow: a system for large-scale machine learning. In: 12th USENIX symposium on operating systems design and implementation (OSDI 16), pp 265–283
19. Paszke A, Gross S, Chintala S, Chanan G, Yang E, DeVito Z, Lin Z, Desmaison A, Antiga L, Lerer A (2017) Automatic differentiation in PyTorch, in NIPS 2017 Workshopon Autodiff. https://openreview.net/forum?id=BJJsrmfCZ
20. Lu L, Meng X, Mao Z, Karniadakis GE (2021) DeepXDE: A deep learning library for solving differential equations. SIAM Rev 63(1):208–228
21. Deng Y, Lin G, Yang X (2020) Multifidelity data fusion via gradient-enhanced Gaussian process regression. arXiv preprint arXiv:2008.01066
22. Yu J, Lu L, Meng X, Karniadakis GE (2022) Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems. Comput Methods Appl Mech Eng 393:114823
23. Mehra M, Kumar V (2007) Fast wavelet-Taylor Galerkin method for linear and non-linear wave problems. Appl Math Comput 189(2):1292–1299

24. Kumar V, Srinivasan B (2019) A novel adaptive mesh strategy for singularly perturbed parabolic convection diffusion problems. Differ Equ Dyn Syst 27(1):203–220
25. Li Jing, Feng Zongcai, Schuster Gerard (2017) Wave-equation dispersion inversion. Geophys J Int 208(3):1567–1578
26. Gu J, Zhang Y, Dong H (2018) Dynamic behaviors of interaction solutions of $(3 + 1)-$ dimensional shallow water wave equation. Comput Math Appl 76(6):1408–1419
27. Kim D (2019) A modified PML acoustic wave equation. Symmetry 11(2):177
28. Kant S, Kumar V (2015) Analysis of an eco-epidemiological model with migrating and refuging prey. In: Mathematical analysis and its applications, Springer, New Delhi, pp 17–36
29. Arora C, Kumar V, Kant S (2017) Dynamics of a high-dimensional stage-structured prey-predator model. Int J Appl Comput Math 3(1):427–445